



Instituto Infnet

Fundamentos do Processamento de Dados

Introdução a Programação com Python [26E1_2]

Funções

Funções são blocos de código reutilizáveis que realizam uma tarefa específica. Elas permitem organizar o código, evitar repetições e facilitar a manutenção.

Funções Integradas (Built-in)

Python possui diversas funções integradas que são sempre disponíveis.

- [Documentação sobre funções integradas](#)

```
#print
print('Olá Mundo!')
```

Olá Mundo!

```
nome = input('Digite seu nome:')
nome
```

'Gesiel Lopes'

```
#funcoes conversao de str numericas em valores numericos
print(int('2'), int('2'))
print(float('2'), float('2'))
```

int 2
float 2.0

```
# funcao type
print(int('2'), type(int('2')))
print(float('2'), type(float('2')))
```

int <class 'int'>
float <class 'float'>

```
# funcao len
nome = 'Gesiel Lopes'
len(nome)
```

12

Sintaxe

```
def nome_da_funcao(parametro1, parametro2, ...):
    """Docstring: documentação da função."""
    # Corpo da função
    # Código que será executado quando a função for chamada

    return valor # Opcional
```

Boas práticas

- Use nomes descritivos para funções (verbos que indiquem ação)
- Siga a convenção snake_case para nomes de funções em Python
- Inclua docstrings para documentar o propósito da função
- Mantenha funções pequenas e focadas em uma única tarefa

```
## exemplo simples de saudação
def saudacao():
    '''Imprime uma saudação simples.'''
    print('Olá, Mundo! Python Eh F***')
```

```
# chamada da funcao
saudacao()
```

Olá, Mundo! Python Eh F***

```
def nova_funcao_de_saudacao():
    '''Definicao de uma nova funcao de saudacao'''
    print('Olá, Mundo! Python Eh F***')
```

nova_funcao_de_saudacao()

Execution error
NameError: name 'nova_funcao_de_saudacao' is not defined

Show error details

```
nova_funcao_de_saudacao()
```

```
Olá, Mundo! Python Eh F***
```

Parâmetros e Argumentos

Parâmetros são variáveis listadas na definição da função. Argumentos são os valores reais passados para a função quando ela é chamada.

```
def somar_dois_numeros(x, y):  
    """  
        Funcao responsavel por realizar a soma de dois valores.  
  
        Argumentos  
        - x: primeiro valor  
        ... - y: segundo valor  
  
        soma = x + y  
  
        print(f'A soma dos valores: {x, y} eh: {soma}')
```

```
somar_dois_numeros(1, 2)
```

```
A soma dos valores: (1, 2) eh: 3
```

```
help(somar_dois_numeros)
```

```
Help on function somar_dois_numeros in module __main__:  
  
somar_dois_numeros(x, y)  
    Funcao responsavel por realizar a soma de dois valores.  
  
    Argumentos  
    - x: primeiro valor  
    - y: segundo valor
```

```
help(print)
```

```
Help on built-in function print in module builtins:  
  
print(*args, sep=' ', end='\n', file=None, flush=False)  
    Prints the values to a stream, or to sys.stdout by default.  
  
    sep  
        string inserted between values, default a space.  
    end  
        string appended after the last value, default a newline.  
    file  
        a file-like object (stream); defaults to the current sys.stdout.  
    flush  
        whether to forcibly flush the stream.
```

```
"""  
Perform clustering on 'X' and returns cluster labels.  
  
Parameters  
-----  
X : array-like of shape (n_samples, n_features)  
    Input data.  
  
y : Ignored  
    Not used, present for API consistency by convention.  
  
**kwargs : dict  
    Arguments to be passed to ``fit``.  
  
    .. versionadded:: 1.4  
  
Returns  
-----  
labels : ndarray of shape (n_samples,), dtype=np.int64  
    Cluster labels.  
"""
```

```
#demonstrando as docstrings dos metodos com o skit-Learn  
from sklearn.base import ClusterMixin  
  
help(ClusterMixin.fit_predict)
```

```
Help on function fit_predict in module sklearn.base:  
  
fit_predict(self, X, y=None)  
    Perform clustering on 'X' and returns cluster labels.  
  
    Parameters  
    -----  
    X : array-like of shape (n_samples, n_features)  
        Input data.  
  
    y : Ignored  
        Not used, present for API consistency by convention.  
  
    Returns  
    -----  
    labels : ndarray of shape (n_samples,), dtype=np.int64  
        Cluster labels.
```

```
# Posicionais
def calcular_imc(nome, idade, altura, peso):

    print(f'Nome: {nome}')
    print(f'Idade: {idade}')

    imc = peso / altura ** 2

    if imc < 17:
        print('Muito abaixo do peso')
    elif 17 <= imc < 18.5:
        print('Abaixo do peso normal')
    elif 18.5 <= imc < 25:
        print('Peso normal')
    elif 25 <= imc < 30:
        print('Acima do peso normal')
    else:
        print('Muito acima do peso normal')
```

```
# parametros posicionais
calcular_imc('Gesiel Lopes', 18, 1.87, 98)
```

Nome: Gesiel Lopes
Idade: 18
Acima do peso normal

```
nome = input('Digite o seu nome:')
idade = int(input('Digite a sua idade:'))
altura = float(input('Digite a sua altura:'))
peso = float(input('Digite o seu peso:'))

calcular_imc(idade, peso, nome, altura)
```

Nome: 18
Idade: 98.0

Execution error

TypeError: unsupported operand type(s) for ** or pow(): 'str' and 'int'

```
-----
TypeError                                Traceback (most recent call last)
Cell In[19], line 6
      3 altura = float(input('Digite a sua altura:'))
      4 peso = float(input('Digite o seu peso:'))
----> 6 calcular_imc(idade, peso, nome, altura)

Cell In[10], line 8, in calcular_imc(nome, idade, altura, peso)
      5 print(f'Nome: {nome}')
      6 print(f'Idade: {idade}')
----> 8 imc = peso / altura ** 2
      9 if imc < 17:
     10     print('Muito abaixo do peso')

TypeError: unsupported operand type(s) for ** or pow(): 'str' and 'int'
```

Hide error details

```
# Nomeados
calcular_imc(18, 98, 'Gesiel Lopes', 1.87)
```

Nome: 18
Idade: 98

Execution error

TypeError: unsupported operand type(s) for ** or pow(): 'str' and 'int'

```
-----
TypeError                                Traceback (most recent call last)
Cell In[22], line 2
      1 # Nomeados
----> 2 calcular_imc(18, 98, 'Gesiel Lopes', 1.87)

Cell In[10], line 8, in calcular_imc(nome, idade, altura, peso)
      5 print(f'Nome: {nome}')
      6 print(f'Idade: {idade}')
----> 8 imc = peso / altura ** 2
      9 if imc < 17:
     10     print('Muito abaixo do peso')

TypeError: unsupported operand type(s) for ** or pow(): 'str' and 'int'
```

Hide error details

```
# parametros nomeados
calcular_imc(idade=18, peso=98, nome='Gesiel Lopes', altura=1.87)
```

Nome: Gesiel Lopes
Idade: 18
Acima do peso normal

help(calcular_imc)

Help on function calcular_imc in module __main__:

calcular_imc(nome, idade, altura, peso)

```
nome = input('Digite o seu nome:')
idade = int(input('Digite a sua idade:'))
altura = float(input('Digite a sua altura:'))
peso = float(input('Digite o seu peso:'))

calcular_imc(idade=idade, peso=peso, nome=nome, altura=altura)
```

Nome: Gesiel Lopes
Idade: 18
Acima do peso normal

Referências

- Documentação oficial sobre funções
- Python PEP 8 - Guia de Estilo